Лабораторийн ажил 5

FoodFlow OpenAPI 3.0 ба кодын жишээ

МУИС, МТЭС, МКУТ, Програм хангамж

Ю.Төгөлдөр, 23B1NUM2350

Таван endpoint ба SRS-ийн холбоо

№	Үйлдэл	Параметр ба request body	Амжилт ба алдаа	W03 шаардлага
1	POST /api/v1/carts/{cart_id}/items	cart_id; food_id, quantity	200 CartItem; 400, 401, 404, 409, 500	FR-04
2	PATCH /api/v1/carts/{cart_id}/items/{item_id}	cart_id, item_id; quantity 1–10	200 CartItem; 400, 401, 404, 409, 500	FR-05
3	PUT /api/v1/orders/{draft_id}/address	draft_id; district, khoroo, street, phone	200 SavedAddress; 400, 401, 404, 409, 500	FR-08
4	PUT /api/v1/orders/{draft_id}/payment-method	draft_id; payment_method	200 SavedPayment; 400, 401, 404, 409, 500	FR-10
5	POST /api/v1/orders	Idempotency-Key header; cart_id, address, payment_method	201 шинэ order, 200 ижил key-ийн давтан хариу; 400, 401, 404, 409, 500	FR-11, NFR-02

Эдгээр нь таван өөр үйлдэл. Read-only GET-д requestBody хүчээр нэмэхгүй гэж сонгож, handout-ийн “бүгд requestBody” шалгуурыг биелүүлэхийн тулд дөрвөн write болон нэг order submit үйлдэл авсан. 4xx, 5xx нь Error schema-д code, message, шаардлагатай бол field гэсэн талбартай. W03-ийн FR-11 шинэ захиалгад HTTP 201-ийг хэлдэг; давтсан key-д 200-г нэмсэн нь ADR-003-ийн idempotent гэрээг уншихад тусалж байна.

YAML эх файл: docs/openapi/openapi.yaml. BearerAuth, CartId/ItemId/DraftId/IdempotencyKey, хүсэлт ба үр дүнгийн schema-ууд components хэсэгт дахин ашиглагдана. Endpoint бүр schema дотроо жишээ утгатай, media type дотор бүрэн request-example болон response-example-тэй. Локал token нь demo-customer-token; бодит хэрэглэгчийн нууц түлхүүр биш.

2 Локал ажиллуулах ба бодит хариуг харах

Сургалтын серверийг repo-ийн үндсэн хавтсаас асаагаад өөр терминалаас таван хүсэлт явуулна. Хүсэлтүүдэд идэвхтэй сагс cart_101, хоол food_id=7, draft_1024, 18,000₮ нэгж үнэ гэсэн локал fixture хэрэглэнэ. Захиалгын нийт үнэ quantity=2 үед 36,000₮ дээр 3,000₮ хүргэлт нэмж 39,000₮ болно.

```
python examples/mock_api.py
# Шинэ терминалд:
bash examples/foodflow_requests.sh
```

Таван FoodFlow жишээ. Эхний дөрөв нь item, quantity, address, payment method-ийг хадгална; тав дахь нь order_id буцаана. Доорх кодыг локал сервер асаасны дараа бүхлээр нь ажиллуулж болно.

```
#!/usr/bin/env bash
set -euo pipefail
api_base='http://127.0.0.1:8765/api/v1'
token='demo-customer-token'

# FR-04: Хоол нэмэх; FoodFlow-ийн сургалтын burger food_id=7.
curl --fail-with-body -sS -X POST "$api_base/carts/cart_101/items" \
  -H "Authorization: Bearer $token" -H 'Content-Type: application/json' \
  -d '{"food_id":7,"quantity":1}'
printf '\n'

# FR-05: Сагсны мөрийг хоёр болгох; subtotal 36,000₮ болно.
curl --fail-with-body -sS -X PATCH "$api_base/carts/cart_101/items/ci_1" \
  -H "Authorization: Bearer $token" -H 'Content-Type: application/json' \
  -d '{"quantity":2}'
printf '\n'

# FR-08: Дөрвөн талбартай хүргэлтийн хаяг хадгалах.
curl --fail-with-body -sS -X PUT "$api_base/orders/draft_1024/address" \
  -H "Authorization: Bearer $token" -H 'Content-Type: application/json' \
  -d '{"district":"Сүхбаатар","khoroo":8,"street":"Бага тойруу 12-5","phone":"99112233"}'
printf '\n'

# FR-10: Захиалгын ноорогт QPay аргыг сонгох.
curl --fail-with-body -sS -X PUT "$api_base/orders/draft_1024/payment-method" \
```

```
  -H "Authorization: Bearer $token" -H 'Content-Type: application/json' \
  -d '{"payment_method":"QPay"}'
printf '\n'

# FR-11: Захиалга үүсгэх; ижил key ба body-г давтахад 200, ижил order_id ирнэ.
curl --fail-with-body -sS -X POST "$api_base/orders" \
  -H "Authorization: Bearer $token" -H 'Idempotency-Key: ff-order-1024' \
  -H 'Content-Type: application/json' \
  -d '{"cart_id":"cart_101","address":{"district":"Сүхбаатар","khoroo":8,"street":"Бага тойруу 12-5","phone":"99112233"},"payment_method":"QPay"}'
printf '\n'
```

Шалгагдсан JSON үр дүн: FR-04-т 18,000₮, FR-05-т 36,000₮, FR-08-д дөрвөн талбартай address, FR-10-д QPay, FR-11-д order_id FF-1024, status Pending, total_mnt 39000 буцсан. Тест нь YAML-ийн таван амжилттай response-example-ийг локал серверийн хариутай тулгаж шалгадаг. Ижил key, ижил body-г давтан илгээхэд 201 → 200 болон ижил body; ижил key, өөр body-д 409 ирсэн. Дотоод тестийн 3 багц амжилттай; production серверийн payload гэж мэдүүлээгүй.

UE-5 Corg.ly-ийн гурван Python кодын жишээ

Corg.ly бол зааварт өгөгдсөн зохиомол дадлагын систем. Нэгэн ижил mock сервер дээр /v1 гэсэн гурван замыг ажиллуулж, жишээний хариуг шалгасан. einstein.jpg нь сургалтын хар цагаан зураг, einstein_bark.wav нь богино синтетик WAV; амьтны жинхэнэ дүрс/дуу гэж нэрлээгүй. Доорх код бүр Python стандарт сан ашиглана.

POST /v1/pets/upload-photo

Эхний жишээ pet_id=corgi_98231-ийн einstein.jpg файлыг multipart-аар илгээнэ. Сервер 201 ба зурагт хандах локал URL буцаана.

```
"""UE-5 sample 1: upload Einstein's training image to the local Corg.ly mock."""
from pathlib import Path
from urllib.request import Request, urlopen
import json

photo_bytes = (Path(__file__).parent / 'data/einstein.jpg').read_bytes()
boundary = 'CorglyEinsteinPhotoBoundary'
multipart_body = (
    f'--{boundary}\r\nContent-Disposition: form-data; name="pet_id"\r\n\r\ncorgi_98231\r\n'
    f'--{boundary}\r\nContent-Disposition: form-data; name="photo"; filename="einstein.jpg"\r\n'
    'Content-Type: image/jpeg\r\n\r\n'
).encode() + photo_bytes + f'\r\n--{boundary}--\r\n'.encode()
request = Request(
    'http://127.0.0.1:8765/v1/pets/upload-photo', data=multipart_body,
    headers={'Authorization':'Bearer demo-corgly-token',
             'Content-Type':f'multipart/form-data; boundary={boundary}'},
    method='POST',
)
with urlopen(request) as response:
    print(response.status, json.dumps(json.load(response), ensure_ascii=False))
```

Шалгалтын үр дүн: 201, pet_id=corgi_98231, status=uploaded. Энэ утга локал mock-ийн бодит хариунаас авсан.

POST /v1/audio/translate-bark

Хоёр дахь жишээ WAV байтыг, pet ID болон en хэлийг илгээнэ. Локал mock 200 ба “I want to play.” гэсэн сургалтын утга буцаана.

```
"""UE-5 sample 2: upload a training WAV and read the local mock translation."""
from pathlib import Path
from urllib.request import Request, urlopen
import json

audio_bytes = (Path(__file__).parent / 'data/einstein_bark.wav').read_bytes()
request = Request(
    'http://127.0.0.1:8765/v1/audio/translate-bark', data=audio_bytes,
    headers={'Authorization':'Bearer demo-corgly-token','Content-Type':'audio/wav',
             'X-Pet-Id':'corgi_98231','X-Target-Language':'en'},
    method='POST',
)
with urlopen(request) as response:
    print(response.status, json.dumps(json.load(response), ensure_ascii=False))
```

Шалгалтын үр дүн: 200, pet_id=corgi_98231, language=en. Энэ утга локал mock-ийн бодит хариунаас авсан.

POST /v1/webhooks/subscribe

Гурав дахь жишээ pet.activity үйл явдлын локал webhook URL-ийг бүртгэнэ. Mock 201 ба sub_corgi_1024 дугаар буцаана; бодит event илгээгээгүй.

```
"""UE-5 sample 3: register a local receiver for pet.activity events."""
from urllib.request import Request, urlopen
import json

subscription = {'url':'http://127.0.0.1:8765/webhook-receiver','events':['pet.activity']}
request = Request(
    'http://127.0.0.1:8765/v1/webhooks/subscribe',
    data=json.dumps(subscription).encode('utf-8'),
    headers={'Authorization':'Bearer demo-corgly-token','Content-Type':'application/json'},
    method='POST',
)
with urlopen(request) as response:
    print(response.status, json.dumps(json.load(response), ensure_ascii=False))
```

Шалгалтын үр дүн: 201, subscription_id=sub_corgi_1024, status=active. Энэ утга локал mock-ийн бодит хариунаас авсан.

Bhatti-ийн таван зарчмаар кодын чанарыг шалгах нь

Explained: код бүрийн өмнө зорилго, оролтыг тайлбарласан. Concise: API-д хамаарах мөрүүдийг үлдээсэн. Clear: cart_id, payment_method гэх утгатай нэр хэрэглэсэн. Usable: repo-д өгөгдөлтэй файл болон ажиллуулах тушаал байна. Trustworthy: 8 хүсэлтийг локал mock-ийн шалгагдсан хариутай тулгасан; production-той хараахан шалгаагүй тул энэ баганыг 4/5 гэж үнэлэв. “foo/bar/test” шиг хийсвэр нэр хүсэлт, хариуны утгад үлдээгүй.

Кодын жишээ	Explained	Concise	Clear	Usable	Trustworthy	Нийт /25
FoodFlow add item	5/5	5/5	5/5	5/5	4/5	24/25
FoodFlow quantity	5/5	5/5	5/5	5/5	4/5	24/25
FoodFlow address	5/5	4/5	5/5	5/5	4/5	23/25
FoodFlow payment	5/5	5/5	5/5	5/5	4/5	24/25
FoodFlow order	5/5	4/5	5/5	5/5	4/5	23/25
Corg.ly photo	5/5	4/5	5/5	5/5	4/5	23/25
Corg.ly translation	5/5	5/5	5/5	5/5	4/5	24/25
Corg.ly webhook	5/5	5/5	5/5	5/5	4/5	24/25

Дундаж: 189/200 = 4.725/5 = 94.5%. 4.0/5.0 буюу 80%-ийн босгыг локал mock-д шалгасан жишээний хувьд давсан. Богино кодын зарчим multipart болон JSON body дээр шаардлагатай мөрийг хасах шалтгаан болоогүй.

Swagger UI ба Redoc

Swagger UI-г багийн дотоод шалгалтад сонгов. Учир нь Try it out товчоор токен, хүсэлт, хариуг нэг дэлгэцээс туршиж болно. Энэ нь Анугийн API-г холбоход хэрэгтэй. Redoc нь гурван хэсэгтэй зохион байгуулалттай тул уншигч endpoint, тайлбар, жишээг зэрэг харахад эвтэйхэн. Олон endpoint дундаас хайж олох хугацааг бодит хэрэглэгчээр хэмжээгүй тул аль нь хурдан гэж дүгнэхгүй. Локал mock-ийн Swagger UI дээр турших урсгалыг, нийтэд унших хувилбарт Redoc-ийг санал болгов. Аль аль нь нэг openapi.yaml файлыг уншина. Жинхэнэ сервер болон нийтэд нээлттэй URL бэлэн болмогц хайлт, уншигдах байдал, Try it out-ийн үр дүнг хэрэглэгчээр шалгаж шийдвэрээ батална. Энэ шийдвэрийг дараагийн sprint-д дахин хянана

Хоёр харагдац нэг public/openapi.yaml файлыг уншина. Локал mock асаалттай үед Swagger UI, Redoc гэсэн хоёр замаар орж болно. Static HTML нь CDN-ийн JS/CSS татдаг тул интернет холболт хэрэгтэй. Public хавтсыг нийтлэхэд хоёр HTML ба YAML-ыг хамтад нь байршуулна; одоогоор нийтэд нээлттэй URL-д байршуулж, браузерт харагдах байдлыг баталгаажуулаагүй. Try it out нь локал сервер асаалттай машин дээр ажиллана.

Шалгалт ба Definition of Done

```
python tests/validate_openapi.py
python -m unittest discover -s tests -v
# Интернеттэй орчинд сургалтын зааврын CLI-г ажиллуулах:
npx --yes @redocly/cli@latest lint docs/openapi/openapi.yaml
```

Шалгуур	Бодит байдал
OpenAPI 3.0.3, 5 endpoint, auth, components, examples	YAML бэлэн; PyYAML parse, бүх local $ref, 5 operations, хүсэлт/хариу шалгасан. Redocly CLI татагдахгүй байсан тул “lint zero errors” гэж мэдүүлээгүй.
Swagger UI, Redoc зэрэг харагдах	Хоёр static HTML, локал mock-ийн хоёр route бэлэн; public URL болон browser render баталгаажаагүй.
Найман sample, 5 зарчмын audit ≥80%	8/8 sample-ийг локал mock-д туршсан; score 94.5%.

Шалгуур	Бодит байдал
100 үгийн тайлан, CI бонус	Сонголтын бичвэр бэлэн; GitHub Actions workflow-д validator, unittest, Redocly lint оруулсан. Hosted CI run хараахан байхгүй.

Standup: таван endpoint-д request/response schema бий; FoodFlow-ийн 5 cURL, Corg.ly-ийн 3 Python жишээг audit-д оруулсан; 4xx/5xx Error schema бүгдэд байна. Retrospective: BearerAuth-ийг W04 ADR-003-аас сонгосон. Хийсвэр утгуудыг cart_101, FF-1024, einstein.jpg зэрэг локал fixture-ээр сольсон. Уншигчийн баримтад Redoc, API-г туршихад Swagger UI илүү тохиромжтой гэж үзэв.

Эх сурвалж: Хавсаргасан Week 05 handout pp. 1–3; Bhatti et al., Ch. 5 pp. 83–99, Ch. 7 pp. 121–130; Chinchilla, Ch. 2 pp. 18–22, Ch. 6 p. 79. OpenAPI 3.0.3, Redocly CLI lint, Swagger UI суулгалт, Redoc HTML element.